

Лабораторна робота №3

Тема: Навчання штучної нейронної мережі

Виконав ст. Групи ПМі-33
Яценко Владислав

Мета роботи: Здобуття практичних умінь у розробці, конфігуруванні та тренуванні каскадних багат шарових нейронних мереж для розпізнавання візуальних образів (символів) за допомогою платформи MATLAB

Завдання за варіантом №20

- Символи для розпізнавання: τ (tau), x, Z, X
- Тип мережі: Каскадна нейронна мережа (newcf) – парний варіант
- Функції активації: logsig (шар 1) + tansig (шар 2)
- Алгоритм навчання: trainlm – метод Левенберга–Марквардта
- Структура навчальної вибірки (20 зображень): для кожного з 4 символів створено еталон та 4 модифікації – зміщене (_S1), повернуте (_R1), пошкоджене (_D1), масштабоване (_C1). Тестова вибірка (16 зображень): відповідні модифікації _S2, _R2, _D2, _C2.

1. Параметри трансформацій

Тип	Навчальна (S1/R1/D1/C1)	Тестова (S2/R2/D2/C2)
Shift	зсув +2 по X, +1 по Y	зсув -1 по X, +2 по Y
Rotate	+ 18°	-12°
Damage	7% шуму	11% шуму
Scale	0.72 (зменшення)	0.85 (зменшення)

2. Програмна реалізація

2.1. p_Read_Vector_1.m

Скрипт зчитує зображення навчальної вибірки з папки Neuro_Train та формує матриці P (256×20) і T (4×20). Кожне зображення розгортається у вектор з 256 елементів (16×16 пікселів).

```
% p_Read_Vector_1.m
% Зчитування зображень навчальної вибірки та формування матриць P і T
% Варіант 20: символи  $\tau$ , x, Z, X (файли X = Xbig)

fid = fopen('Neuro_Train/File_Name_tx1.txt', 'r');
files = textscan(fid, '%d %s');
fclose(fid);

num_images = length(files{1}); % 20
num_classes = 4; %  $\tau$ , x, Z, X

P = zeros(256, num_images);
T = zeros(num_classes, num_images);

for i = 1:num_images
    fname = fullfile('Neuro_Train', [files{2}{i}, '.bmp']);
    img = imread(fname);
    if size(img, 3) == 3
        img = rgb2gray(img);
    end
    P(:, i) = im2double(img(:));
    T(files{1}(i), i) = 1;
end

fprintf('OK: P(%dx%d), T(%dx%d)\n', size(P,1),size(P,2),size(T,1),size(T,2));
```

2.2. Neuro_Image_Train.m (частина коду

Скрипт створює каскадну нейронну мережу (newcf) з двома прихованими шарами (16 і 12 нейронів), функціями активації logsig та tansig, та алгоритмом навчання trainlm. Навчання проводиться послідовно для трьох значень goal: 10^{-2} , 10^{-3} , 10^{-4} . Результати зберігаються у mat-файли.

```

for g = 1:3
    goalVal = goals(g);

    net = newcf(P, T, [16 12], {'logsig','tansig'});
    net.divideFcn = '';
    net.trainFcn = 'trainlm';
    net.performFcn = 'mse';
    net.trainParam.epochs = 1500;
    net.trainParam.goal = goalVal;
    net.trainParam.time = 120;
    net.trainParam.showWindow = true;
    net = init(net);

    fprintf('\n=== Навчання %d/3: goal = %s ===\n', g, goalNames{g});
    tic;
    [net, tr] = train(net, P, T);
    t = toc;

    resGoal(g) = goalVal;
    resEpoch(g) = tr.num_epochs;
    resTime(g) = t;
    resMSE(g) = tr.best_perf;

    saveName = ['net_lm_' goalNames{g} '.mat'];
    save(saveName, 'net', 'tr', 't');
    fprintf('Збережено: %s | Epoch: %d | Час: %.3f с | MSE: %.5e\n', ...
        saveName, tr.num_epochs, t, tr.best_perf);

    semilogy(tr.epoch, tr.perf, '-', 'Color', colors{g}, ...
        'LineWidth', 2, 'DisplayName', ['goal=' goalNames{g}]);
end

```

2.3. Neuro_Image_Recog.m(частина коду)

Скрипт завантажує навчені мережі та виконує розпізнавання 16 тестових зображень з папки Neuro_Recogn. Для кожного зображення виводиться вихідний вектор Y та визначається клас символу за максимальним значенням виходу.

```

for i = 1:numTests
    fname = fullfile('Neuro_Recogn', [testFiles{i} '.bmp']);
    img = imread(fname);
    if size(img,3)==3, img = rgb2gray(img); end
    Y = net(im2double(img(:)));
    Yall(:,i) = Y;

    [~, pred] = max(Y);
    ok = (pred == trueLabels(i));
    if ok, correct = correct+1; end

    yStr = sprintf('%.3f ', Y);
    fprintf('%-14s | Y=[%s] | %-8s | очік: %-8s | %s\n', ...
        testFiles{i}, yStr, classNames{pred}, ...
        classNames{trueLabels(i)}, ...
        char('✓'*ok + 'X'*(1-ok)));
end

```

2.4. Neuro_Exp_V.m(частина коду)

Скрипт реалізує статистичний експеримент методом Монте-Карло. Мережа навчається 7 разів для кожного з трьох значень QV1 (8, 16, 24), фіксується час навчання, після чого розраховуються статистичні показники: середнє значення t_c , відхилення σ , межа похибки δ та мінімальна кількість випробувань $m1$.

```
for qi = 1:3
    QV1 = qv1_vals(qi);
    t_k = zeros(1, m);

    fprintf('%s:\n', labels{qi});
    for k = 1:m
        net_e = newcf(P, T, [QV1 12], {'logsig','tansig'});
        net_e.divideFcn = '';
        net_e.trainFcn = 'trainlm';
        net_e.trainParam.epochs = 1500;
        net_e.trainParam.goal = 1e-3;
        net_e.trainParam.time = 120;
        net_e.trainParam.showWindow = false;
        net_e = init(net_e);

        tic;
        train(net_e, P, T);
        t_k(k) = toc;
        fprintf('  k=%d: %.4f c\n', k, t_k(k));
    end

    tc = mean(t_k);
    sigma = sqrt(sum((t_k - tc).^2) / m);
    s = sigma * sqrt(m/(m-1));
    delta = t_gamma * s / sqrt(m);
    m1 = ceil((t_gamma * s / delta1)^2);
```

3. Приклади тестових зображень

Для перевірки ефективності роботи нейромережі було створено вибірки світлин розміром 16x16 пікселів. Нижче наведено приклади еталонного та змінених зображень символу x (зміщення, обертання, шум):



Рис. 3.1: Еталонне та модифіковані зображення символу x

4. Результати експериментів

4.1. Експеримент А: Вплив значення goal на навчання

```

=== Навчання 1/3: goal = 1e-02 ===
Збережено: net_lm_1e-02.mat | Епох: 3 | Час: 3.482 с | MSE: 9.99231e-04

=== Навчання 2/3: goal = 1e-03 ===
Збережено: net_lm_1e-03.mat | Епох: 3 | Час: 2.149 с | MSE: 1.67611e-04

=== Навчання 3/3: goal = 1e-04 ===
Збережено: net_lm_1e-04.mat | Епох: 4 | Час: 2.126 с | MSE: 3.87268e-06

Goal      Епохи    Час (с)    MSE
-----
1e-02     3        3.482     9.99231e-04
1e-03     3        2.149     1.67611e-04
1e-04     4        2.126     3.87268e-06

```

Рис. 4.1. Результати навчання для трьох значень goal

Табл. 4.1. Результати навчання ШНМ для різних значень goal

Goal (MSE)	Кількість епох	Час навчання, с	Фінальна MSE
1e-02	3	3.482	9.99e-04
1e-03	3	2.149	1.68e-04
1e-04	4	2.126	3.87e-06

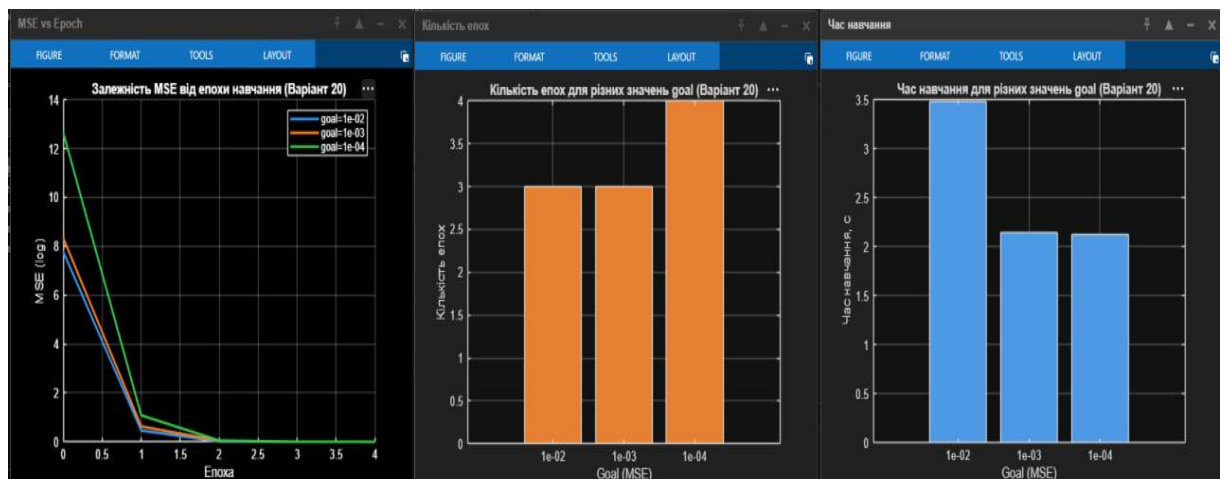


Рис. 4.2. Графіки MSE vs Епоха, кількість епох та час навчання

```

=====
РОЗПІЗНАВАННЯ: goal = 1e-02
=====
tau1_S2      | Y=[-2.939 0.579 1.859 0.992 ] | Z      | очік:  $\tau$  (tau) | X
tau1_R2      | Y=[2.234 -0.203 -0.116 0.265 ] |  $\tau$  (tau) | очік:  $\tau$  (tau) | ✓
tau1_D2      | Y=[-0.255 -0.738 1.442 0.419 ] | Z      | очік:  $\tau$  (tau) | X
tau1_C2      | Y=[0.130 0.571 -0.162 -0.589 ] | x      | очік:  $\tau$  (tau) | X
x1_S2        | Y=[0.395 2.319 2.035 3.025 ] | X      | очік: x        | X
x1_R2        | Y=[2.409 -0.894 0.628 1.428 ] |  $\tau$  (tau) | очік: x        | X
x1_D2        | Y=[0.110 -0.505 2.017 0.088 ] | Z      | очік: x        | X
x1_C2        | Y=[-0.283 0.953 -0.669 -0.011 ] | x      | очік: x        | ✓
Z1_S2        | Y=[0.979 -2.054 -1.315 3.499 ] | X      | очік: Z        | X
Z1_R2        | Y=[1.656 -1.524 0.482 -1.342 ] |  $\tau$  (tau) | очік: Z        | X
Z1_D2        | Y=[-3.576 0.663 -1.292 -0.063 ] | x      | очік: Z        | X
Z1_C2        | Y=[3.940 0.024 2.066 -3.064 ] |  $\tau$  (tau) | очік: Z        | X
Xbig1_S2     | Y=[2.813 2.562 1.158 1.525 ] |  $\tau$  (tau) | очік: X        | X
Xbig1_R2     | Y=[0.544 -2.622 3.733 -0.278 ] | Z      | очік: X        | X
Xbig1_D2     | Y=[0.124 -2.751 2.421 0.156 ] | Z      | очік: X        | X
Xbig1_C2     | Y=[0.820 -0.280 -0.540 1.350 ] | X      | очік: X        | ✓
>>> Точність: 3/16 = 18.8%

```

Рис. 4.3. Розпізнавання при goal = 1e-02 (точність 18.8%)

```

=====
РОЗПІЗНАВАННЯ: goal = 1e-03
=====
tau1_S2      | Y=[1.878 1.629 -0.122 -2.130 ] |  $\tau$  (tau) | очік:  $\tau$  (tau) | ✓
tau1_R2      | Y=[0.537 0.281 -2.418 -1.267 ] |  $\tau$  (tau) | очік:  $\tau$  (tau) | ✓
tau1_D2      | Y=[-0.461 -1.124 0.998 -0.523 ] | Z      | очік:  $\tau$  (tau) | X
tau1_C2      | Y=[0.636 -0.726 0.258 0.799 ] | X      | очік:  $\tau$  (tau) | X
x1_S2        | Y=[3.489 2.884 0.090 -2.123 ] |  $\tau$  (tau) | очік: x        | X
x1_R2        | Y=[-0.074 -0.259 1.005 -0.367 ] | Z      | очік: x        | X
x1_D2        | Y=[2.826 1.284 -0.967 0.795 ] |  $\tau$  (tau) | очік: x        | X
x1_C2        | Y=[-0.810 0.071 0.361 0.229 ] | Z      | очік: x        | X
Z1_S2        | Y=[3.836 2.030 0.592 -3.644 ] |  $\tau$  (tau) | очік: Z        | X
Z1_R2        | Y=[-0.028 -3.908 -0.955 -0.608 ] |  $\tau$  (tau) | очік: Z        | X
Z1_D2        | Y=[0.546 1.096 2.032 0.322 ] | Z      | очік: Z        | ✓
Z1_C2        | Y=[0.734 -0.412 1.189 -0.135 ] | Z      | очік: Z        | ✓
Xbig1_S2     | Y=[-0.146 1.229 0.579 -2.869 ] | x      | очік: X        | X
Xbig1_R2     | Y=[1.857 -0.179 -0.072 1.596 ] |  $\tau$  (tau) | очік: X        | X
Xbig1_D2     | Y=[2.141 -0.528 -2.008 1.596 ] |  $\tau$  (tau) | очік: X        | X
Xbig1_C2     | Y=[-1.094 -1.524 -0.459 0.682 ] | X      | очік: X        | ✓
>>> Точність: 5/16 = 31.2%

```

Рис. 4.4. Розпізнавання при goal = 1e-03 (точність 31.2%)


```

=====
РОЗПІЗНАВАННЯ: goal = 1e-04
=====
tau1_S2      | Y=[1.028 2.022 0.489 -0.060 ] | x      | очік: τ (tau) | X
tau1_R2      | Y=[2.074 -0.093 -0.401 0.453 ] | τ (tau) | очік: τ (tau) | ✓
tau1_D2      | Y=[0.734 2.644 -2.139 -3.288 ] | x      | очік: τ (tau) | X
tau1_C2      | Y=[1.252 -0.869 -0.194 -0.727 ] | τ (tau) | очік: τ (tau) | ✓
x1_S2        | Y=[0.684 0.994 0.322 -0.819 ] | x      | очік: x       | ✓
x1_R2        | Y=[-0.536 0.288 -1.293 -0.186 ] | x      | очік: x       | ✓
x1_D2        | Y=[-0.882 2.121 1.422 1.565 ] | x      | очік: x       | ✓
x1_C2        | Y=[-0.013 -0.109 0.146 0.304 ] | X      | очік: x       | X
Z1_S2        | Y=[-1.713 0.791 2.559 1.368 ] | Z      | очік: Z       | ✓
Z1_R2        | Y=[0.748 4.235 -0.298 -2.310 ] | x      | очік: Z       | X
Z1_D2        | Y=[3.533 5.477 0.582 -0.449 ] | x      | очік: Z       | X
Z1_C2        | Y=[-0.175 -0.440 -1.672 -0.023 ] | X      | очік: Z       | X
Xbig1_S2     | Y=[-1.507 0.836 0.666 -4.636 ] | x      | очік: X       | X
Xbig1_R2     | Y=[-1.733 4.266 0.778 2.329 ] | x      | очік: X       | X
Xbig1_D2     | Y=[-2.779 -3.636 -2.865 -3.438 ] | τ (tau) | очік: X       | X
Xbig1_C2     | Y=[-0.097 -0.452 -0.636 0.989 ] | X      | очік: X       | ✓
>>> Точність: 7/16 = 43.8%

```

Рис. 4.5. Розпізнавання при goal = 1e-04 (точність 43.8%)

Табл. 4.2. Зведена таблиця точності розпізнавання

Goal (MSE)	Правильно	Неправильно	Точність, %
1e-02	3/16	13/16	18.8
1e-03	5/16	11/16	31.2
1e-04	7/16	9/16	43.8

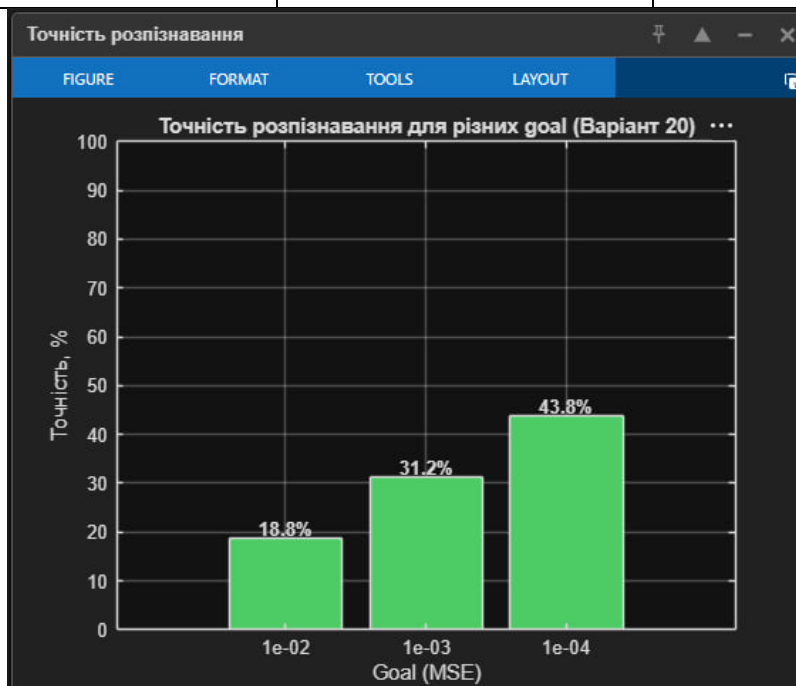


Рис. 4.6. Точність розпізнавання для різних значень goal

4.2. Експеримент Б: Статистичний аналіз методом Монте-Карл

Для парного варіанту змінюється кількість нейронів у першому прихованому шарі QV1. Навчання проводилось $m = 7$ разів для кожного значення QV1 при $\text{goal} = 10^{-3}$

```

=== Експеримент Б: Монте-Карло (Варіант 20) ===

QV1=8 (-50%):
k=1: 0.5768 c
k=2: 0.7364 c
k=3: 0.9196 c
k=4: 0.5854 c
k=5: 0.7544 c
k=6: 0.7318 c
k=7: 0.7372 c
→ tc=0.7202, σ=0.1071, δ=0.1072, m1=9

QV1=16 (база):
k=1: 1.5580 c
k=2: 1.2445 c
k=3: 1.4481 c
k=4: 1.5172 c
k=5: 1.5605 c
k=6: 1.5547 c
k=7: 1.4800 c
→ tc=1.4804, σ=0.1042, δ=0.1042, m1=8

QV1=24 (+50%):
k=1: 2.7377 c
k=2: 2.6830 c
k=3: 2.6339 c
k=4: 2.0322 c
k=5: 2.7396 c
k=6: 2.6302 c
k=7: 2.6308 c
→ tc=2.5839, σ=0.2296, δ=0.2296, m1=37

QV1          | t_k (c)          | tc | σ | δ | m1
=====
QV1=8 (-50%) | 0.58 0.74 0.92 0.59 0.75 0.73 0.74 | 0.7202 | 0.1071 | 0.1072 | 9
QV1=16 (база) | 1.56 1.24 1.45 1.52 1.56 1.55 1.48 | 1.4804 | 0.1042 | 0.1042 | 8
QV1=24 (+50%) | 2.74 2.68 2.63 2.03 2.74 2.63 2.63 | 2.5839 | 0.2296 | 0.2296 | 37

```

Рис. 4.7. Результати Монте-Карло: значення $t(k)$ та статистика

Табл. 4.3. Результати статистичного аналізу за методом Монте-Карло

QV1	$t(k)$, c	t_c , c	σ	δ	$m1$
8 (-50%)	0.58 0.74 0.92 0.59 0.75 0.73 0.74	0.7202	0.1071	0.1072	9
16 (база)	1.56 1.24 1.45 1.52 1.56 1.55 1.48	1.4804	0.1042	0.1042	8
24 (+50%)	2.74 2.68 2.63 2.03 2.74 2.63 2.63	2.5839	0.2296	0.2296	37

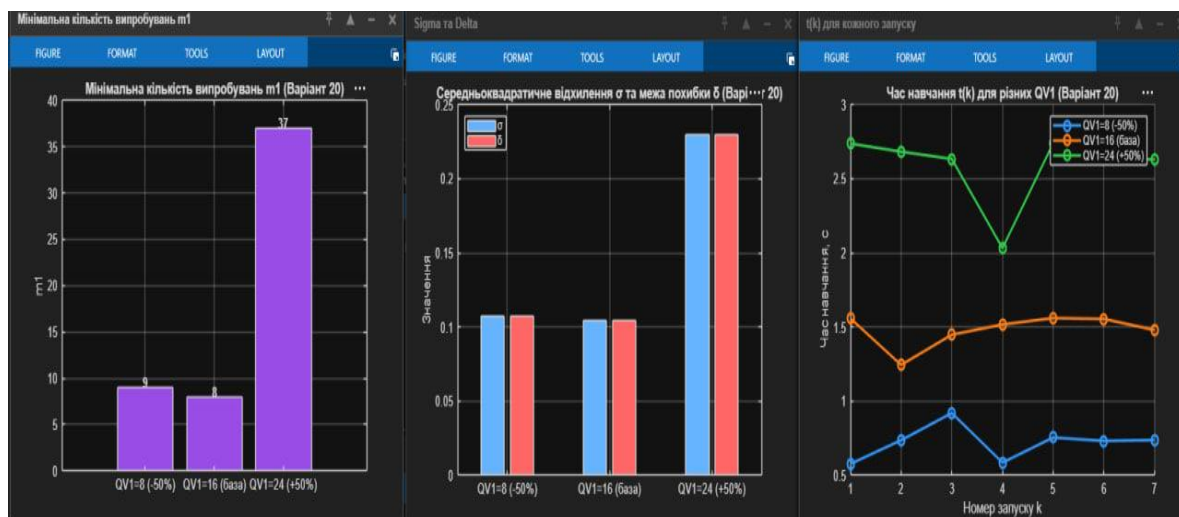


Рис. 4.8. Графіки: $t(k)$, σ/δ , $m1$ для різних QV1

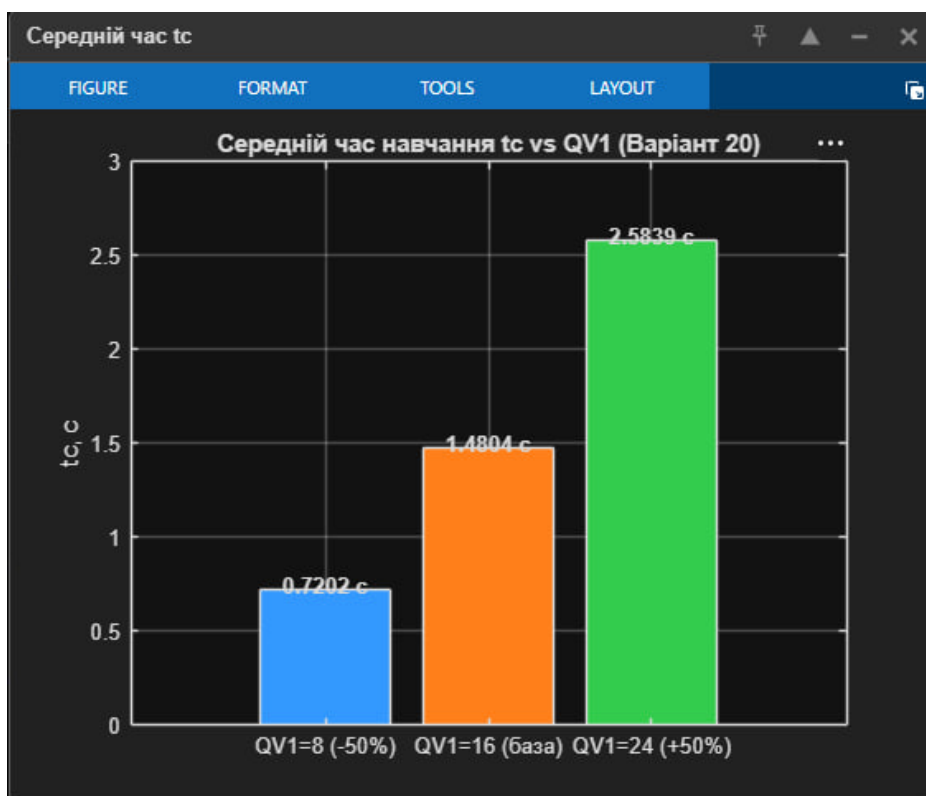


Рис. 4.9. Середній час навчання t_c для різних значень QV1

5. Висновки

1. Алгоритм Левенберга–Марквардта (trainlm) продемонстрував надзвичайно високу швидкість збіжності – мережа виходила на задану точність уже після 3–4 ітерацій навчання. Це пояснюється тим, що метод використовує наближення другого порядку (матриця Гессе), що дозволяє робити значно точніші кроки оптимізації порівняно зі звичайним градієнтним спуском.

2. Зі зменшенням порогового значення похибки goal спостерігається покращення якості розпізнавання: при $goal=10^{-2}$ вдалося правильно класифікувати лише 3 зображення з 16 (18.8%), тоді як при $goal=10^{-4}$ – вже 7 з 16 (43.8%). Втім, загальна точність залишається невисокою, що зумовлено малою роздільністю вхідних зображень та різницею між навчальними і тестовими трансформаціями.

3. Аналіз результатів розпізнавання показав, що мережа краще справляється з масштабованими та слабко повернутими зображеннями, оскільки вони зберігають основну форму символу. Натомість пошкоджені зображення з випадковим шумом розпізнаються найгірше – шум суттєво змінює розподіл яскравості пікселів, який є вхідною ознакою мережі.

4. Результати статистичного дослідження підтвердили, що збільшення кількості нейронів у першому прихованому шарі QV1 призводить до зростання часу навчання: середній час збільшився майже вчетверо при переході від QV1=8 до QV1=24. Водночас зросла і нестабільність результатів – значення σ для QV1=24 у два рази вище ніж для QV1=8.

5. Розраховане мінімальне число випробувань $m1$ показало, що для архітектур з QV1=8 і QV1=16 семи запусків достатньо для отримання надійної статистики ($\gamma=0.95$). Для QV1=24 теоретично потрібно 37 запусків через більшу варіативність часу навчання, що пов'язано з більшим простором параметрів та чутливістю до початкової ініціалізації ваг.